Practical 5: Regression Models## Practical Information| Attribute | Details ||---------|---------|| **Practical Number** | 5 || **Title** | Regression Models || **Duration** | 4 Hours || **Course Outcomes (COs)** | CO2, CO4 || **Description** | Implement and evaluate different regression models for predictive analysis. |## Key Topics Covered- Linear Regression- Polynomial Regression- Ridge/Lasso- SVR- Ensemble methods---## Instructions for Students1. **Complete all tasks** outlined in this practical2. **Write your code** in the cells provided below3. **Add comments** to explain your code4. **Include proper output** (prints, visualizations, etc.)5. **Submit your completed notebook** before the deadline---## Your Work Starts HereUse the cell(s) below to write your implementation:

```
# Practical 5: Regression Models
# Write your implementation here

# Practical 5: Regression Models
# Write your implementation here

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset (Regression dataset)
data = load_diabetes()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("✅ Dataset loaded and scaled successfully!\n")

# ---------------- LINEAR REGRESSION ----------------
linear_model = LinearRegression()
linear_model.fit(X_train_scaled, y_train)
y_pred_lr = linear_model.predict(X_test_scaled)

# ---------------- POLYNOMIAL REGRESSION ----------------
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)

poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)
y_pred_poly = poly_model.predict(X_test_poly)

# ---------------- RIDGE REGRESSION ----------------
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train_scaled, y_train)
```

```
ridge_model.fit(X_train_scaled, y_train)
y_pred_ridge = ridge_model.predict(X_test_scaled)

# ---------------- LASSO REGRESSION ----------------
lasso_model = Lasso(alpha=0.01)
lasso_model.fit(X_train_scaled, y_train)
y_pred_lasso = lasso_model.predict(X_test_scaled)

# ---------------- SUPPORT VECTOR REGRESSION ----------------
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train_scaled, y_train)
y_pred_svr = svr_model.predict(X_test_scaled)

# ---------------- RANDOM FOREST REGRESSION ----------------
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)   # RF does not need scaling
y_pred_rf = rf_model.predict(X_test)

# ---------------- METRICS FUNCTION ----------------
def print_metrics(name, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"📊 {name} Performance:")
    print(f"   MSE  : {mse:.2f}")
    print(f"   RMSE : {rmse:.2f}")
    print(f"   MAE  : {mae:.2f}")
    print(f"   R²   : {r2:.4f}\n")

# Print Results
print_metrics("Linear Regression", y_test, y_pred_lr)
print_metrics("Polynomial Regression", y_test, y_pred_poly)
print_metrics("Ridge Regression", y_test, y_pred_ridge)
print_metrics("Lasso Regression", y_test, y_pred_lasso)
print_metrics("SVR", y_test, y_pred_svr)
print_metrics("Random Forest Regression", y_test, y_pred_rf)

print("✅ Practical 5 completed successfully!")
```

```
✅ Dataset loaded and scaled successfully!

📊 Linear Regression Performance:
   MSE  : 2900.19
   RMSE : 53.85
   MAE  : 42.79
   R²   : 0.4526

📊 Polynomial Regression Performance:
   MSE  : 3096.03
   RMSE : 55.64
   MAE  : 43.58
   R²   : 0.4156

📊 Ridge Regression Performance:
   MSE  : 2892.01
   RMSE : 53.78
   MAE  : 42.81
   R²   : 0.4541

📊 Lasso Regression Performance:
   MSE  : 2898.37
   RMSE : 53.84
   MAE  : 42.80
   R²   : 0.4529

📊 SVR Performance:
   MSE  : 4332.74
   RMSE : 65.82
   MAE  : 56.03
   R²   : 0.1822

📊 Random Forest Regression Performance:
   MSE  : 2952.01
   RMSE : 54.33
   MAE  : 44.05
   R²   : 0.4428

✅ Practical 5 completed successfully!
```

```
# Additional code cell

# Practical 5: Regression Models
# Write your implementation here

import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
data = load_diabetes()
X = data.data
y = data.target

# Simple Regression (using one feature: BMI)
X_simple = X[:, 2].reshape(-1, 1)  # BMI feature

# Train-test split
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)

# Train Simple Linear Regression
simple_model = LinearRegression()
simple_model.fit(X_train_s, y_train_s)
y_pred_simple = simple_model.predict(X_test_s)

# Multiple Regression (all features)
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_m = scaler.fit_transform(X_train_m)
X_test_m = scaler.transform(X_test_m)

multi_model = LinearRegression()
multi_model.fit(X_train_m, y_train_m)
y_pred_multi = multi_model.predict(X_test_m)

# Metrics function
def show_metrics(title, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"\n📊 {title}")
    print(f"MSE  : {mse:.2f}")
    print(f"RMSE : {rmse:.2f}")
    print(f"MAE  : {mae:.2f}")
    print(f"R²   : {r2:.4f}")

# Results
show_metrics("Simple Linear Regression (BMI)", y_test_s, y_pred_simple)
show_metrics("Multiple Linear Regression (All Features)", y_test_m, y_pred_multi)

print("\n✅ Practical 5 completed successfully!")
```

```
📊 Simple Linear Regression (BMI)
MSE  : 4061.83
RMSE : 63.73
MAE  : 52.26
R²   : 0.2334

📊 Multiple Linear Regression (All Features)
MSE  : 2900.19
RMSE : 53.85
MAE  : 42.79
R²   : 0.4526

✅ Practical 5 completed successfully!
```

```
# Additional code cell

# Practical 5: Regression Models
# Write your implementation here

import numpy as np
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
data = load_diabetes()
X = data.data
y = data.target

# Use only one feature for visualization (BMI)
X = X[:, 2].reshape(-1, 1)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Polynomial Regression
poly = PolynomialFeatures(degree=3)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)
y_pred_poly = poly_model.predict(X_test_poly)

# Support Vector Regression (SVR)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svr_model = SVR(kernel='rbf')
svr_model.fit(X_train_scaled, y_train)
y_pred_svr = svr_model.predict(X_test_scaled)

# Metrics function
def print_metrics(name, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"\n📊 {name}")
    print(f"MSE  : {mse:.2f}")
    print(f"RMSE : {rmse:.2f}")
    print(f"MAE  : {mae:.2f}")
    print(f"R²   : {r2:.4f}")

# Print Results
print_metrics("Polynomial Regression (Degree 3)", y_test, y_pred_poly)
print_metrics("Support Vector Regression (SVR)", y_test, y_pred_svr)

print("\n✅ Practical 5 completed successfully!")
```

```
📊 Polynomial Regression (Degree 3)
MSE  : 4064.44
RMSE : 63.75
MAE  : 52.18
R²   : 0.2329

📊 Support Vector Regression (SVR)
MSE  : 4256.45
RMSE : 65.24
MAE  : 55.19
R²   : 0.1966
```

```
✅ Practical 5 completed successfully!
```